Merkhilfe sympy

ktheu.github.io/sympy-mathe

Start

```
from sympy import *
init_printing()                          Formelausgabe
x, y = symbols('x y')
symbols('x', real=True)                  auch positive, integer
display(a, b)                            mehrere Ausgaben
N(w)  N(w, 5)  w.evalf(10)               Dezimalzahl
pi  E  I  oo                             π, e, i, ∞
```

Bis Klasse 10

1 Zahlen exakt

sympy_zahlen

```
Rational(1, 3)  Rational('0.25')         Bruch
Integer(1) / 3                           statt 1/3
isprime(n)  factorint(n)                 Primfaktoren
primefactors(n)  divisors(n)
divisor_count(n)  prime(k)
nextprime(n)  primerange(a, b)
gcd(a, b)  lcm(a, b)                     ggT, kgV
sqrt(8)  root(x, 3)                      Wurzeln
radsimp(1/sqrt(2))                       Nenner rational
nsimplify(0.75)                          Dezimal → Bruch
```

2 Terme

sympy_terme

```
t.subs(x, 2)  t.subs({x: 1, y: 2})
expand(t)  factor(t)
collect(t, x)  simplify(t)
cancel(t)  together(t)                   Bruchterme
fraction(t)  denom(t)                    Zähler, Nenner
powsimp(t)  expand_power_base(t)
binomial(n, k)                           (n k)
Eq(links, rechts)                        Gleichung
simplify(a - b) == 0                     gleichwertig?
```

3 Gleichungen

sympy_gleichungen

```
solve(Eq(l, r), x)                       Liste
solve([g1, g2], [x, y])                  Dictionary
solveset(g, x, S.Reals)                  Menge
solveset(g, x, domain=Interval(0, 2*pi))
Interval(a, b)  Interval.open(a, b)
Interval.Ropen(a, b)                     [a; b[
linsolve([g1, g2], [x, y])               LGS
nsolve(g, x, 1.5)                        numerisch
log(x)  log(x, 2)                        ln, log_2
ceiling(w)  floor(w)                     auf-, abrunden
```

4 Funktionen

sympy_funktionen

```
f = x**2 - 1  f.subs(x, 3)
plot(f, g, (x, -3, 3), ylim=(-2, 5),
  legend=True, title='...');
maximum(f, x, Interval(a, b))  minimum
from sympy.calculus.util import function_range
function_range(f, x, S.Reals)            Wertemenge
continuous_domain(f, x, S.Reals)
limit(f, x, oo)  limit(f, x, 0, '+')
degree(p, x)  roots(p, x)                mit Vielfachheit
sin  cos  tan  periodicity(f, x)
rad(60)  deg(pi/3)                       Grad ↔ Bogenmaß
```

5 Ableitungen

sympy_ableitungen

```
diff(f, x)  diff(f, x, 2)
limit((f.subs(x, x0 + h) - f.subs(x, x0)) / h, h, 0)
is_increasing(f, Interval(a, b))  is_decreasing
Abs(w)  atan(m)                          Steigungswinkel
```

6 Geometrie

sympy_geometrie

```
asin  acos  atan  trigsimp(t)
Point(0, 0)  Segment(A, B)  Line(A, B)
Circle(M, r)  Triangle(A, B, C)
intersection(g, h)  A.distance(B)
T.area  T.angles  T.centroid
T.circumcenter  T.circumradius  T.incenter
Segment(A, B).midpoint
v = Matrix([1, 2, 3])                    Vektor
v.norm()  v.cross(w)                     Länge, Kreuzprodukt
Point3D(1, 2, 3)  Line3D(P, Q)
```

7 Wahrscheinlichkeit

sympy_wahrscheinlichkeit

```
FiniteSet(1, 2, 3)  A | B  A & B
Complement(Omega, A)  ProductSet(W, W)
factorial(n)  ff(n, k)  binomial(n, k)
from sympy.stats import *                E, P neu belegt
Die('X', 6)  FiniteRV('X', {0: p, 5: q})
Binomial('X', n, p)
P(X > 4)  P(Eq(X, 2))  P(X <= 3)
E(X)  variance(X)  std(X)
density(X).dict                          Verteilung
```

Kursstufe

8 Ableitungsregeln

sympy_ableitungsregeln

```
exp(x)  log(x)                           e^x, ln x
expand_log(t, force=True)
logcombine(t, force=True)
limit(f, x, 2, '+-')                     beidseitig
```

9 Funktionsanalyse

sympy_funktionsanalyse

```
a = symbols('a', positive=True)          Schar
l = solve([g1, g2, g3], [a, b, c])  f.subs(l)
solve(..., dict=True)
plot(*[f.subs(a, w) for w in [1, 2, 3]], (x, 0, 5));
plot(..., aspect_ratio=(1, 1), xlim=(a, b))
```

10 Integrale

sympy_integrale

```
integrate(f, x)                          Stammfunktion
integrate(f, (x, a, b))  (x, 0, oo)
Integral(f, (x, a, b))  .doit()
```

11 Wachstum

sympy_wachstum

```
f = Function('f')
dgl = Eq(f(t).diff(t), k * f(t))
l = dsolve(dgl, f(t), ics={f(0): 100})  l.rhs
checkodesol(dgl, l)                      (True, 0)
```

12 Ebenen

sympy_ebenen

```
u.dot(v)                                 Skalarprodukt
M = Matrix([[1, 1, 1, 6], [2, -1, 1, 3]])
M.rref()[0]  M.det()  linsolve(M, [x, y, z])
M.row(1)  M[1, :] = M.row(1) - 2 * M.row(0)
E = Plane(Point3D(1, 0, 2), normal_vector=(1, -1, 1))
Plane(A, B, C)  E.equation(x, y, z)
E.intersection(g)                        Gerade, Ebene
from sympy.plotting import plot3d
```

13 Abstände und Winkel

sympy_abstaende_winkel

```
deg(acos(u.dot(v) / (u.norm() * v.norm())))
E.distance(P)  E.projection(P)           Lotfußpunkt
E.angle_between(F)
Line3D(P, direction_ratio=(1, 2, 2))
g.distance(h)                            auch windschief
```

14 Hypothesentest

sympy_hypothesentest

```
F = cdf(Binomial('X', n, p))             F[k] = P(X ≤ k)
1 - F[k - 1]                             P(X ≥ k)
```

15 Normalverteilung

sympy_normalverteilung

```
X = Normal('X', mu, sigma)
P((X >= a) & (X <= b))
density(X)(x)  cdf(X)(x)
quantile(X)(Rational(9, 10))
fl = lambdify(x, f)                      für matplotlib
```

Merkhilfe sympy

Vertiefungskurs

16 Logik und Beweise

sympy_logik_beweise

```
from sympy.logic.boolalg import truth_table
A & B   A | B   ~A   Xor(A, B)
Implies(A, B)   Equivalent(A, B)
truth_table(e, [A, B])   simplify_logic(e)
satisfiable(e)
summation(k**2, (k, 1, n))
```

17 Folgen und Reihen

sympy_folgen_reihen

```
sequence(1/n, (n, 1, oo))[:5]
rsolve(f(n + 1) - 2*f(n), f(n), {f(1): 3})
limit(a, n, oo)   fibonacci(n)
summation(q**k, (k, 0, oo))
Sum(1/k**2, (k, 1, oo)).is_convergent()
apart(t, k)                                  Partialbrüche
```

18 Komplexe Zahlen

sympy_komplexe_zahlen

```
z = 3 + 4*I   expand(z1 * z2)
re(z)   im(z)   conjugate(z)
Abs(z)   arg(z)
expand_complex(2 * exp(I * pi / 3))
factor(p, extension=I)
```

19 Integrationstechniken

sympy_integrationstechniken

```
integrate(u, u).subs(u, g)                   Substitution
solve(..., simplify=False)
```

Wahlfach Digitale Mathematische Werkzeuge

20 Gleichungen, Kurven

sympy_dmw_gleichungen

```
real_roots(p)   count_roots(p)
discriminant(p, x)   real_root(-8, 3)
N(w, 5, chop=True)                           ohne Rest-i
Poly(p, x).degree()
kurve(Eq(x**2 + y**2, 1), xb=(-2, 2), yb=(-2, 2))   Setup
```

21 Krümmung

sympy_dmw_kruemmung

```
plot_parametric(cos(t), sin(t), (t, 0, 2*pi))
p = plot(f, (x, a, b), show=False)
p.extend(q)   p.show()
Circle(P1, P2, P3)
nsolve(t, a, (0, 1), solver='bisect')
```

22 Flächen im Raum

sympy_dmw_flaechen

```
from sympy.plotting import plot3d, plot_contour
plot3d(f, (x, -2, 2), (y, -2, 2))
plot_contour(f, (x, -2, 2), (y, -2, 2))
diff(f, x, y)   hessian(f, (x, y))
Matrix([diff(f, x), diff(f, y)])             Gradient
atan2(y, x)
```

23 Taylor

sympy_dmw_taylor

```
series(f, x, 0, 5)   .removeO()
series(f, x, a, 4)                           um x = a
Piecewise((exp(-1/x), x > 0), (0, True))
```

24 Differentialgleichungen

sympy_dmw_dgl

```
richtungsfeld(x - y, xb=(-3, 3), yb=(-3, 3))   Setup
dsolve(dgl(x * y), Y(x), ics={Y(0): 1})        Setup
ics={Y(0): 1, Y(x).diff(x).subs(x, 0): 0}
f = lambdify((x, y), x - y)
```

Zeichnen mit matplotlib

```
import matplotlib.pyplot as plt
plt.plot(xs, ys, 'o-')   plt.bar(xs, ys)
plt.step(xs, ys, where='post')
plt.fill_between(xs, ys, alpha=0.4)
plt.quiver(xs, ys, us, vs)
plt.axhline(0)   plt.axvline(0)
plt.gca().set_aspect('equal')
plt.xlabel('x')   plt.legend()   plt.show()
```